
Oliver Krauss

Modern Code - Recursion vs. Iteration

ModernCode | MTD.ba VZ SS25

Hagenberg 09.12.2025

HAGENBERG | LINZ | STEYR | WELS



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

Recursion vs. Iteration

- **Two fundamental approaches** to solving repetitive problems
- **Every programmer faces this choice:** How should I structure my solution?
- **Real-world impact:** Understanding both makes you a better problem solver
- **Connection to what you know:**
 - Methods (Session 04): Recursion uses method calls
 - Sorting (Session 07): Merge Sort and Quick Sort use recursion
 - Control structures (Session 03): Iteration uses loops
- **The key question:** When should I use recursion, and when should I use iteration?

Why This Matters

- **Problem-solving flexibility:** Some problems are naturally recursive, others iterative
- **Performance awareness:** Understanding trade-offs helps you write efficient code
- **Debugging skills:** Knowing how each approach works helps you find bugs faster
- **Code readability:** Choosing the right approach makes code clearer
- **Industry relevance:** Both patterns appear in real-world software
- **Foundation for advanced topics:** Trees, graphs, and complex algorithms build on these concepts

Before We Begin: Think About This

Can you think of problems that naturally break into smaller versions of themselves?

Before We Begin: Examples to Consider

Real-World Recursive Problems

- **Factorial**: Calculating factorial of 5 = $5 \times$ factorial of 4
- **Sorting**: Sorting an array = sort left half + sort right half + merge
- **Search**: Finding an item = check middle, then search left or right half

Problems Outside Software Engineering

- **Family tree**: Finding ancestors = find parents, then find their parents ...
- **Russian nesting dolls**: Each doll contains a smaller version of itself
- **Reading files**: Read all files in folder, then read all files in subfolders ...

What patterns do you notice?

What is Recursion?

- **Definition:** A method that calls itself to solve a problem
- **Key insight:** Break a problem into smaller instances of the same problem
- **Two essential components:**
 - **Base case:** The stopping condition (prevents infinite recursion)
 - **Recursive case:** The method calls itself with a smaller problem
- **Mathematical connection:** Similar to mathematical induction

Recursion: The Two Essential Parts

Recursive Case

Base Case

- The **simplest** version of the problem
- Can be solved **directly** without recursion
- **Stops** the recursion from continuing
- Example: Factorial of 0 or 1
- Example: Array with 0 or 1 element

- Breaks problem into **smaller** version
- Calls the **same method** with smaller input
- **Trusts** the recursive call to solve the smaller problem
- Example: $\text{Factorial}(n) = n \times \text{Factorial}(n-1)$
- Example: Sort array = sort halves + merge

The Pattern:

1. Check if we're at the base case → return result
2. Otherwise, solve a smaller problem recursively
3. Combine the result with current step

Simple Recursive Example: Countdown I

```
1  static void countdown(int n) {
2      // Base case: stop when n <= 0
3      if (n <= 0) {
4          IO.println("Blast off!");
5          return;
6      }
7      // Recursive case: print n, then countdown(n-1)
8      IO.println(n);
9      countdown(n - 1);
10 }
```


Simple Recursive Example: Countdown II

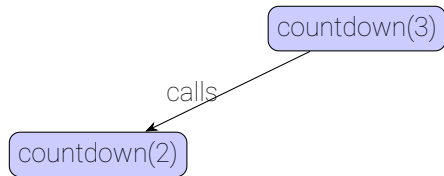
How it works:

- `countdown(3)` prints 3, then calls `countdown(2)`
- `countdown(2)` prints 2, then calls `countdown(1)`
- `countdown(1)` prints 1, then calls `countdown(0)`
- `countdown(0)` hits base case, prints "Blast off!" and returns

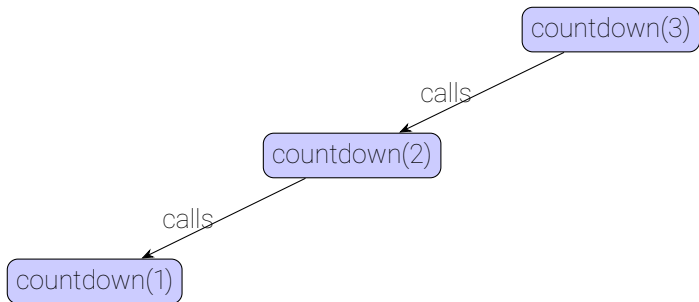
Recursive Call Tree: Countdown

countdown(3)

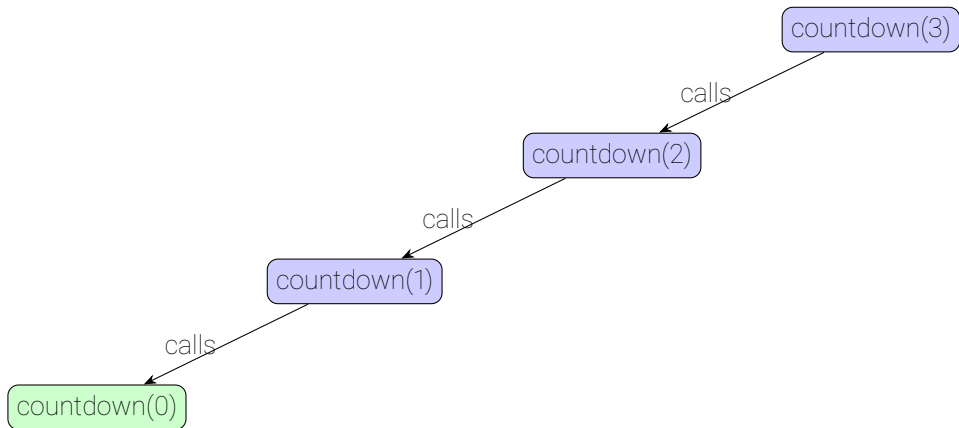
Recursive Call Tree: Countdown



Recursive Call Tree: Countdown

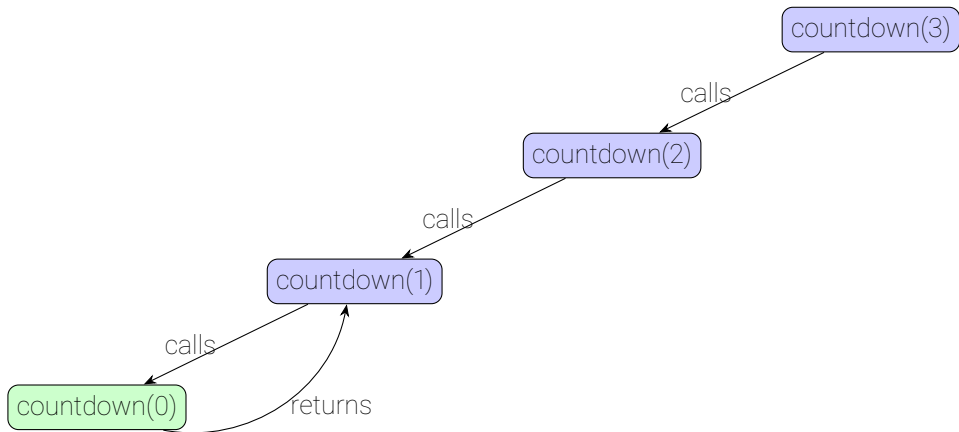


Recursive Call Tree: Countdown



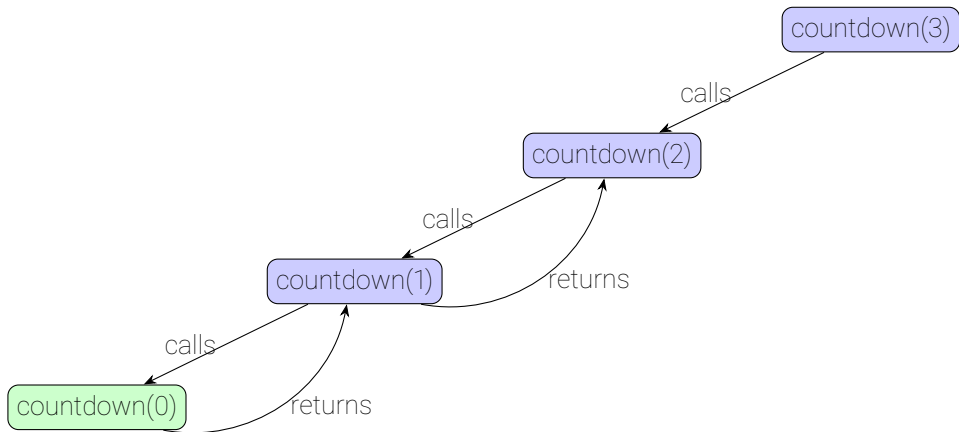
Base case - stops recursion

Recursive Call Tree: Countdown



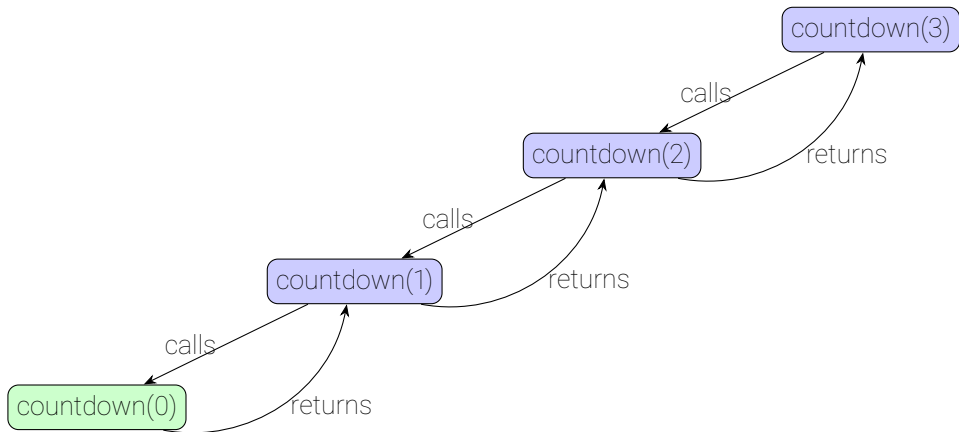
Base case - stops recursion

Recursive Call Tree: Countdown



Base case - stops recursion

Recursive Call Tree: Countdown



Base case - stops recursion

Output: 3, 2, 1, "Blast off!"

Recursive Example: Factorial

- **Mathematical definition:** $n! = n \times (n - 1) \times (n - 2) \times \dots \times 1$
- **Recursive definition:** $n! = n \times (n - 1)!$ where $0! = 1$ and $1! = 1$

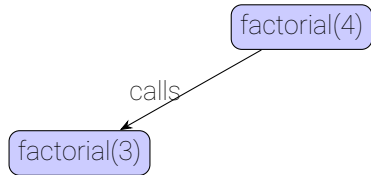
```
1  static int factorial(int n) {  
2      // Base case: 0! = 1 and 1! = 1  
3      if (n <= 1) {  
4          return 1;  
5      }  
6      // Recursive case: n! = n  $\times$  (n-1)!  
7      return n * factorial(n - 1);  
8  }
```

Example: $\text{factorial}(4) = 4 \times \text{factorial}(3) = 4 \times 3 \times \text{factorial}(2) = 4 \times 3 \times 2 \times \text{factorial}(1) = 4 \times 3 \times 2 \times 1 = 24$

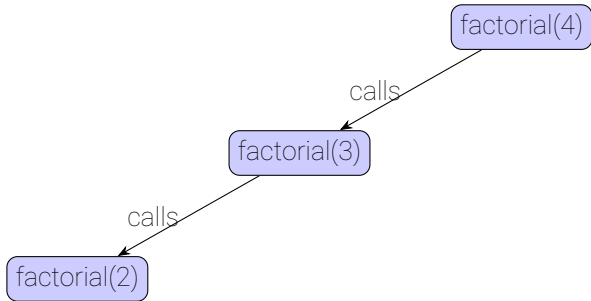
Recursive Call Tree: Factorial(4)

factorial(4)

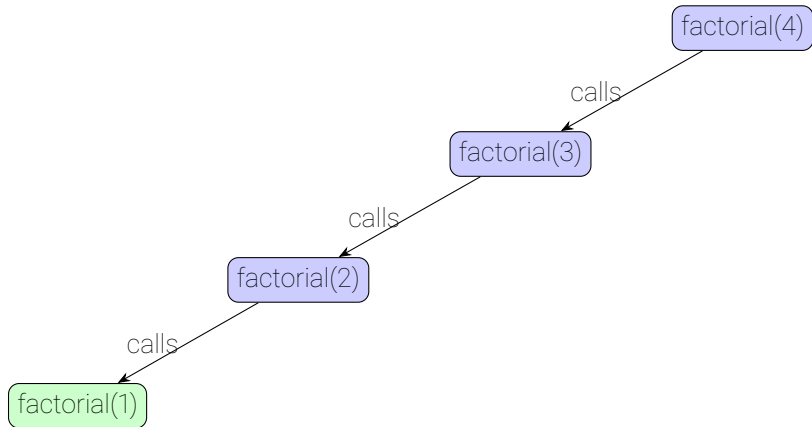
Recursive Call Tree: Factorial(4)



Recursive Call Tree: Factorial(4)

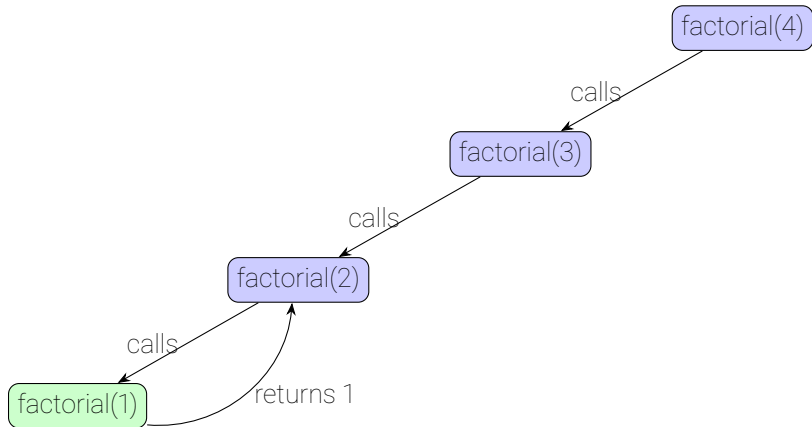


Recursive Call Tree: Factorial(4)



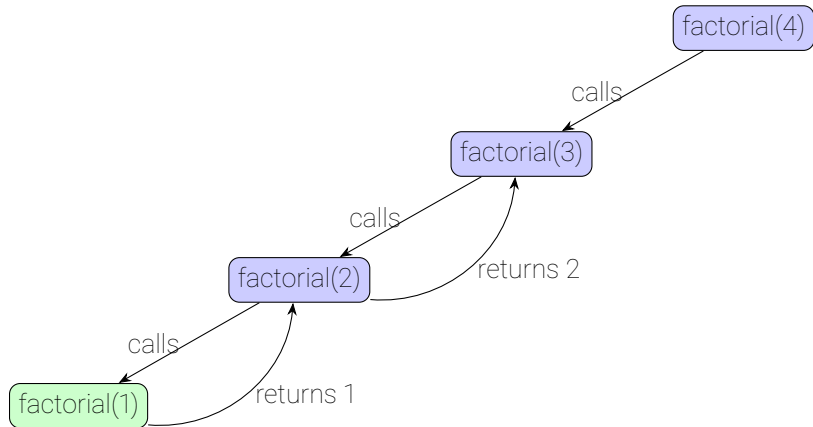
Base case - returns 1

Recursive Call Tree: Factorial(4)



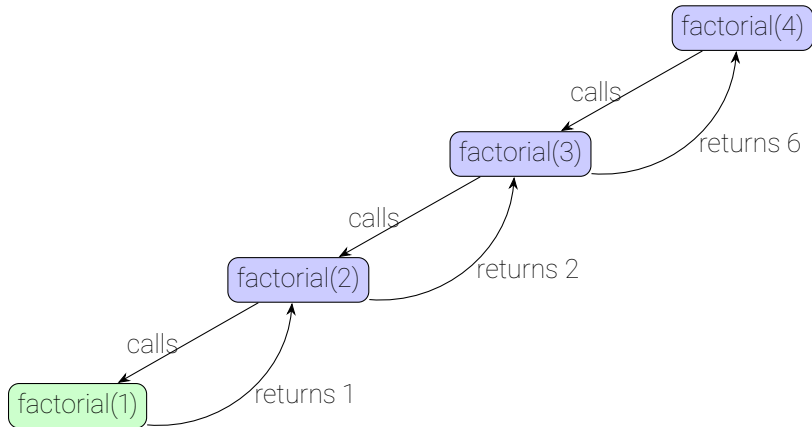
Base case - returns 1

Recursive Call Tree: Factorial(4)



Base case - returns 1

Recursive Call Tree: Factorial(4)



Base case - returns 1

Result: factorial(4) = 24

Recursion & Mathematical Induction

- **Mathematical induction** proves statements for all natural numbers
- **Base case:** Prove statement is true for $n = 0$ or $n = 1$
- **Inductive step:** Assume true for n , prove for $n+1$
- **Recursion mirrors this:**
 - Base case: Solve the simplest problem directly
 - Recursive step: Assume smaller problem is solved, solve current problem

Example: Factorial

- **Base:** $1! = 1$ (proven directly)
- **Inductive:** If $(n - 1)!$ is correct, then $n! = n \times (n - 1)!$ is correct
- **Conclusion:** Factorial works for all positive integers

What is Iteration?

- **You've been using it all the time!**
 - **for** loops, **while** loops, **do-while** loops
 - Processing arrays element by element
 - Repeating actions until a condition is met
- **The word "iterator"** comes from iteration
- **Key insight:** Use control structures to repeat operations
- **State management:** Use variables to track progress through iterations

What is Iteration?

- **Definition:** Repeating a block of code using loops
- **Key insight:** Use control structures to repeat operations
- **Three main loop types:**
 - **for:** Counted loops (known number of iterations)
 - **while:** Conditional loops (unknown number of iterations)
 - **do-while:** Post-test loops (run at least once)
- **State management:** Use variables to track progress through iterations
- **Analogy:** Like walking steps - you repeat the same action, but your position changes

Iteration: The Loop Structure

Initialization

- Set up starting values
- Declare loop variables
- Prepare for first iteration

Condition

- Check if loop should continue
- Evaluated before each iteration
- Must eventually become false

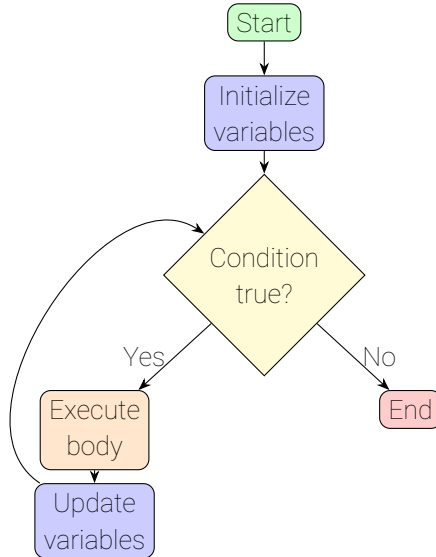
Update

- Modify loop variables
- Move toward termination
- Happens after each iteration

Body

- The work to be repeated
- Executes if condition is true
- Can use loop variables

Iterative Flow Diagram



Iterative Example: Countdown

```
1  static void countdown(int n) {  
2      // Iterative approach: use a loop  
3      for (int i = n; i > 0; i--) {  
4          IO.println(i);  
5      }  
6      IO.println("Blast off!");  
7  }
```

Same output as recursive version: 3, 2, 1, "Blast off!"

Iterative Example: Factorial

```
1  static int factorial(int n) {  
2      int result = 1;  
3      for (int i = 1; i <= n; i++) {  
4          result = result * i;  
5      }  
6      return result;  
7  }
```

-- Initialize: **result** = 1, **i** = 1

-- Iteration 1: **result** = $1 \times 1 = 1$, **i** = 2

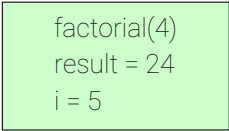
-- Iteration 2: **result** = $1 \times 2 = 2$, **i** = 3

-- Iteration 3: **result** = $2 \times 3 = 6$, **i** = 4

-- Iteration 4: **result** = $6 \times 4 = 24$, **i** = 5

-- Condition false: **i** > **n**, exit loop, return 24

Iterative Factorial: Call Stack Visualization



```
factorial(4)
result = 24
i = 5
```

State changes:

Initial: result=1, i=1 →

Iter 1: result=1, i=2 →

Iter 2: result=2, i=3 →

Iter 3: result=6, i=4 →

Iter 4: result=24, i=5 → Done: return 24

Only ONE stack frame!

Variables change
inside the same
frame

No call stack buildup
No stack unwinding

Key difference: All computation happens in one method call, constant stack space

Iteration: State Management

- **Explicit state**: Variables track progress through iterations
- **Accumulators**: Variables that build up results (e.g., **result** in factorial)
- **Counters**: Variables that track position (e.g., **i** in loops)
- **Advantage**: All state is visible in one place
- **Disadvantage**: Need to manage state manually

Example: Sum of array

- `int sum = 0;` (accumulator)
- `for (int i = 0; i < arr.length; i++)` (counter)
- `sum += arr[i];` (update accumulator)

Recursion vs Iteration:

Same Problem, Different Approaches

Both solve the same problem, but with different mental models

Recursive Thinking

- "Solve a smaller problem"
- "Trust the recursion"
- "Combine results"
- Top-down approach

Iterative Thinking

- "Repeat this action"
- "Update state each time"
- "Accumulate results"
- Bottom-up approach

Factorial: Recursive vs Iterative

Recursive

```
1 static int factorial(int n) {  
2     if (n <= 1) {  
3         return 1;  
4     }  
5     return n * factorial(n - 1);  
6 }
```

- Breaks into smaller problem
- Uses call stack
- More concise

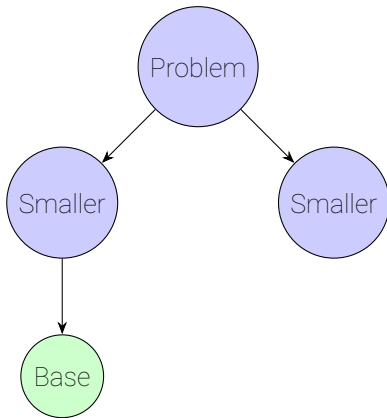
Iterative

```
1 static int factorial(int n) {  
2     int result = 1;  
3     for (int i = 1; i <= n; i++) {  
4         result *= i;  
5     }  
6     return result;  
7 }
```

- Builds up result
- No call stack
- More explicit

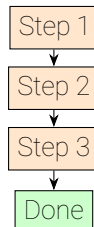
Mental Model Comparison

Recursive Mental Model



"Divide and conquer"

Iterative Mental Model



"Step by step"

Code Structure Comparison

Aspect	Recursion	Iteration
Structure	Method calls itself	Loop repeats code
Termination	Base case check	Loop condition
State	Implicit (parameters)	Explicit (variables)
Memory	Call stack grows	Constant stack usage
Readability	Can be elegant	Usually explicit
Debugging	Harder (call stack)	Easier (single frame)

The Call Stack: Foundation of Recursion (Reminder)

- **What is the stack?** Memory area that stores method call information
- **Stack frame:** Each method call creates a frame containing:
 - Parameters passed to the method
 - Local variables
 - Return address (where to go when method finishes)
- **Stack behavior:** Last In, First Out (LIFO)
- **Recursion uses the stack:** Each recursive call adds a new frame

Stack Growth: Recursive Factorial(4)

factorial(4): n=4, return ?

**Stack grows
as we recurse**



Stack Growth: Recursive Factorial(4)

**Stack grows
as we recurse**

factorial(4): n=4, return ?

factorial(3): n=3, return ?



Stack Growth: Recursive Factorial(4)

**Stack grows
as we recurse**

factorial(4): n=4, return ?

factorial(3): n=3, return ?

factorial(2): n=2, return ?



Stack Growth: Recursive Factorial(4)

**Stack grows
as we recurse**

factorial(4): n=4, return ?

factorial(3): n=3, return ?

factorial(2): n=2, return ?

factorial(1): n=1, return 1



Each recursive call adds a new frame to the stack

Stack Unwinding: Recursive Factorial(4)

**Stack unwinds
as we return**

factorial(4): n=4, return ?

factorial(3): n=3, return ?

factorial(2): n=2, return ?

factorial(1): returns 1



Stack Unwinding: Recursive Factorial(4)

**Stack unwinds
as we return**

factorial(4): n=4, return ?

factorial(3): n=3, return ?

factorial(2): returns $2 \times 1 = 2$



Stack Unwinding: Recursive Factorial(4)

**Stack unwinds
as we return**

factorial(4): n=4, return ?

factorial(3): returns $3 \times 2 = 6$



Stack Unwinding: Recursive Factorial(4)

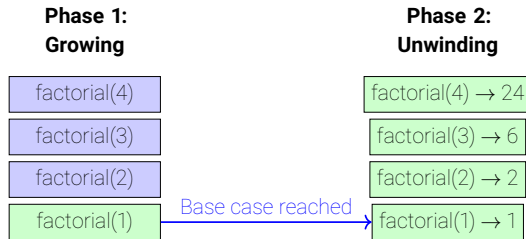
factorial(4): returns $4 \times 6 = 24$

**Stack unwinds
as we return**



Each return removes a frame and passes result up

Complete Stack Lifecycle: Factorial(4)



Stack Overflow: When Recursion Goes Too Deep

- **Stack overflow:** Occurs when call stack exceeds available memory
- **Causes:**
 - Missing or incorrect base case
 - Recursion depth too large
 - Infinite recursion
- **Error message:** `StackOverflowError`
- **Prevention:** Always ensure base case is reachable

Example: Infinite Recursion

`factorial(n)` without base case $\rightarrow$ calls `factorial(n-1)` forever $\rightarrow$ stack overflow

Memory Comparison: Recursion vs Iteration

Recursive Factorial(n)

- **Stack frames:** n frames
- **Memory per frame:** ~ 24 bytes (parameters, return address)
- **Total stack memory:** $O(n)$
- **Example:** factorial(1000) needs ~ 24 KB stack

Iterative Factorial(n)

- **Stack frames:** 1 frame
- **Memory per frame:** ~ 16 bytes (variables)
- **Total stack memory:** $O(1)$
- **Example:** factorial(1000) needs ~ 16 bytes

Key insight: Recursion uses more memory because it stores state for each recursive call

Before We Continue: Think About Trade-offs

Question: Advantages and Disadvantages

What are the advantages and disadvantages of recursion versus iteration?

Before We Continue: Think About Trade-offs

Question: Advantages and Disadvantages

What are the advantages and disadvantages of recursion versus iteration?

Consider: performance, memory, readability, debugging, when each approach works best...

Recursion: Benefits

- **Elegant code:** Often more concise and readable
- **Natural fit:** Some problems are inherently recursive
 - Tree traversals
 - Divide-and-conquer algorithms
 - Mathematical definitions
- **Less state management:** Parameters handle state automatically
- **Mathematical clarity:** Mirrors mathematical definitions
- **Divide-and-conquer:** Naturally breaks problems into subproblems

Example: Merge Sort is naturally recursive - divide array, sort halves, merge

Recursion: Drawbacks

- **Stack overhead:** Each call uses stack memory
- **Performance cost:** Method call overhead
- **Stack overflow risk:** Deep recursion can crash
- **Harder to debug:** Multiple stack frames to trace
- **Less intuitive:** Some developers find it harder to understand
- **Memory usage:** $O(n)$ stack space for depth n

Example: Recursive Fibonacci is very inefficient due to repeated calculations

Iteration: Benefits

- **Performance:** Usually faster (no method call overhead)
- **Memory efficient:** Constant stack space $O(1)$
- **Easy to debug:** Single stack frame, visible state
- **No stack overflow:** Limited only by available memory
- **Intuitive:** Most developers find loops easier to understand
- **Explicit control:** Clear loop conditions and updates

Example: Iterative factorial is straightforward and efficient

Iteration: Drawbacks

- **More verbose:** Sometimes requires more code
- **State management:** Need to manually track variables
- **Less natural:** Some problems don't fit iterative patterns well
- **Complex logic:** Can become hard to read with nested loops
- **Error-prone:** Off-by-one errors, incorrect loop conditions

Example: Iterative tree traversal requires explicit stack management

Performance Comparison

Aspect	Recursion	Iteration
Time complexity	Same (usually)	Same (usually)
Space complexity	$O(n)$ stack	$O(1)$ stack
Method call overhead	Yes	No
Memory usage	Higher	Lower
Speed	Slower (overhead)	Faster

Note: For simple problems like factorial, iteration is usually faster. For complex problems like tree traversal, the difference may be negligible.

Fibonacci: The Classic Example

- **Mathematical definition:**
 - $F(0) = 0, F(1) = 1$ (base cases)
 - $F(n) = F(n - 1) + F(n - 2)$ for $n > 1$ (recursive case)
- **Sequence:** 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...
- **Perfect example** to compare recursion and iteration
- **Shows the problem** with naive recursion (repeated calculations)

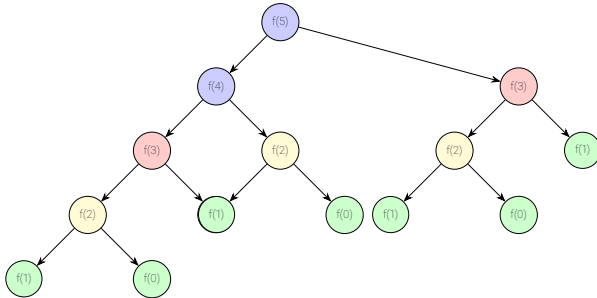
Fibonacci: Recursive Implementation

```
1  static int fibonacci(int n) {  
2      // Base cases  
3      if (n <= 0) return 0;  
4      if (n == 1) return 1;  
5  
6      // Recursive case  
7      return fibonacci(n - 1) + fibonacci(n - 2);  
8  }
```

Problem: This implementation is very inefficient!

- Calculates the same values multiple times
- Example: **fibonacci(5)** calculates **fibonacci(3)** twice

Fibonacci Recursive Call Tree: fibonacci(5)



Problem:

f(3) calculated

twice!

f(2) calculated

three times!

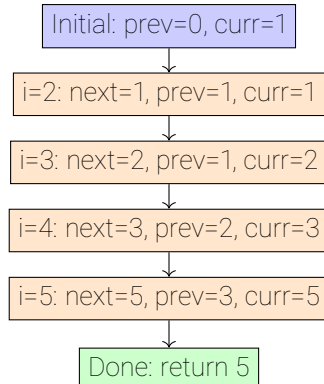
Fibonacci: Iterative Implementation

```
1  static int fibonacci(int n) {  
2      if (n <= 0) return 0;  
3  
4      int prev = 0;          // F(0)  
5      int curr = 1;          // F(1)  
6  
7      for (int i = 2; i <= n; i++) {  
8          int next = prev + curr;  
9          prev = curr;  
10         curr = next;  
11     }  
12     return curr;  
13 }
```

Advantages:

- Time complexity: $O(n)$ - linear!
- Space complexity: $O(1)$ - constant!
- No repeated calculations
- Much faster for large n

Fibonacci Iterative: Step-by-Step (n=5)



Each value calculated exactly once - $O(n)$ time, $O(1)$ space

Fibonacci: Performance Comparison

n	Recursive Calls	Iterative Steps
5	15 calls	4 steps
10	177 calls	9 steps
20	21,891 calls	19 steps
30	2,692,537 calls	29 steps

Key insight: Naive recursion is exponentially slower! Iteration is much more efficient.

Sorting Connection: Merge Sort (Session 07)

- **Recall:** Merge Sort uses recursion naturally
- **Strategy:** Divide array in half, sort each half recursively, merge results
- **Why recursion fits:**
 - Problem naturally breaks into smaller versions
 - Each half is sorted the same way
 - Base case: array with 0 or 1 element (already sorted)

Merge Sort Structure

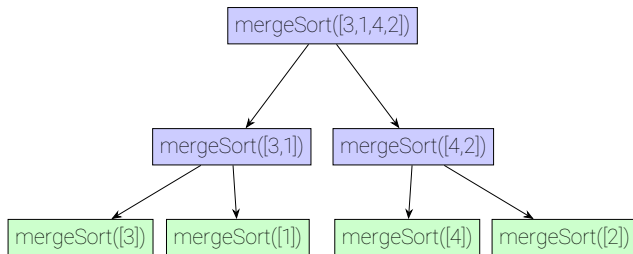
`mergeSort(array) = mergeSort(leftHalf) + mergeSort(rightHalf) + merge()`

Merge Sort: Main Function

```
1 void mergeSort(int[] arr, int left, int right) {  
2     if (left < right) {  
3         int mid = (left + right) / 2;  
4         mergeSort(arr, left, mid);  
5         mergeSort(arr, mid + 1, right);  
6         merge(arr, left, mid, right);  
7     }  
8 }
```

- **Base case:** If $\text{left} \geq \text{right}$, array has 0 or 1 elements (already sorted)
- **Divide:** Split array into two halves at middle position
- **Conquer:** Recursively sort both halves
- **Combine:** Merge the sorted halves back together

Merge Sort: Recursive Call Structure

**Base cases:**

Single elements
(already sorted)

Recursive:

Divide and
conquer

Merge Sort: Setup Phase

```
1 void merge(int[] arr, int left, int mid, int right) {  
2     // Create temporary arrays  
3     int[] leftArr = new int[mid - left + 1];  
4     int[] rightArr = new int[right - mid];  
5  
6     // Copy data to temp arrays  
7     for (int i = 0; i < leftArr.length; i++) {  
8         leftArr[i] = arr[left + i];  
9     }  
10    for (int j = 0; j < rightArr.length; j++) {  
11        rightArr[j] = arr[mid + 1 + j];  
12    }
```

Merge Sort: Merge Phase

```
13      // Merge the temp arrays back
14      int i = 0, j = 0, k = left;
15      while (i < leftArr.length && j < rightArr.length) {
16          if (leftArr[i] <= rightArr[j]) {
17              arr[k] = leftArr[i];
18              i++;
19          } else {
20              arr[k] = rightArr[j];
21              j++;
22          }
23          k++;
24      }
25
```

Merge Sort: Copy Elements

```
25      // Copy remaining elements
26      while (i < leftArr.length) arr[k++] = leftArr[i++];
27      while (j < rightArr.length) arr[k++] = rightArr[j++];
28  }
29
```

Why Recursion Fits Divide-and-Conquer

- **Natural decomposition**: Problems break into identical subproblems
- **Base case clarity**: Smallest problem is easy to solve
- **Combination step**: Results from subproblems combine naturally
- **Code elegance**: Recursive code mirrors the problem structure

Examples of divide-and-conquer:

- Merge Sort: Sort halves, then merge
- Quick Sort: Partition, then sort partitions
- Binary Search: Search half, then search that half

Decision Framework: When to Use Recursion?

Use recursion when:

- **Problem is naturally recursive:** Tree/graph structures, divide-and-conquer
- **Mathematical definition is recursive:** Factorial, Fibonacci, mathematical sequences
- **Code clarity matters more than performance:** Elegant solution is worth small overhead
- **Problem depth is limited:** Won't cause stack overflow
- **State management is complex:** Recursion handles it automatically

Examples: Tree traversal, Merge Sort, Quick Sort, parsing nested structures

Decision Framework: When to Use Iteration?

Use iteration when:

- **Performance is critical**: Need maximum speed
- **Memory is limited**: Can't afford stack overhead
- **Problem depth is very large**: Risk of stack overflow
- **Simple repetitive task**: No need for complex decomposition
- **Debugging ease matters**: Need to trace execution easily

Examples: Summing arrays, factorial, Fibonacci, simple loops

Problem Characteristics: Recursion-Friendly

Recursion-Friendly Problems

- **Tree structures**: Each node has similar structure
- **Divide-and-conquer**: Problem splits into similar subproblems
- **Backtracking**: Need to explore multiple paths
- **Nested structures**: Data with recursive definition
- **Mathematical recursion**: Defined recursively in math

Key pattern: Problem structure mirrors recursive structure

Problem Characteristics: Iteration-Friendly

Iteration-Friendly Problems

- **Linear processing:** Process items in sequence
- **Accumulation:** Build result step by step
- **Simple repetition:** Same action multiple times
- **State machines:** Clear state transitions
- **Performance-critical:** Need maximum efficiency

Key pattern: Problem is sequential, not hierarchical

Hybrid Approaches

- **Recursive with iterative base:** Use recursion for structure, iteration for base cases
- **Iterative with recursive thinking:** Use iterative implementation of recursive algorithm
- **Memoization:** Cache recursive results to avoid repeated calculations
- **Tail recursion:** Optimize recursive calls that don't need stack frames

Example: Iterative Merge Sort (possible but more complex) or Memoized Fibonacci (recursive but efficient)

Real-World Examples

Recursion in Practice

- File system traversal
- XML/JSON parsing
- Compiler parsing
- Tree algorithms
- Graph algorithms

Iteration in Practice

- Array processing
- Database queries
- Game loops
- Data processing pipelines
- Simple algorithms

Common Mistake 1: Missing Base Case

```
1  static int factorial(int n) {  
2      // Missing base case!  
3      return n * factorial(n - 1);  
4  }
```

What happens:

- Infinite recursion: `factorial(5)` calls `factorial(4)`, which calls `factorial(3)`, ...
- Eventually calls `factorial(-1000)`, `factorial(-1001)`, ...
- Stack overflow error: `StackOverflowError`

The fix: Always include a base case that stops the recursion

Common Mistake 2: Incorrect Base Case

```
1  static int factorial(int n) {  
2      if (n == 0) {  
3          return 0;  // Wrong! Should return 1  
4      }  
5      return n * factorial(n - 1);  
6  }
```

- **factorial(0)** returns 0 (should be 1)
- **factorial(1)** = $1 \times \text{factorial}(0)$ = $1 \times 0 = 0$ (wrong!)
- **factorial(2)** = $2 \times 0 = 0$ (wrong!)
- All results are 0!

The fix: Ensure base case returns the correct value

Common Mistake 3: Not Moving To Base Case

```
1  static int factorial(int n) {  
2      if (n <= 1) return 1;  
3      return n * factorial(n);  // Wrong! Should be n-1  
4  }
```

What happens:

- `factorial(5)` calls `factorial(5)` again
- Infinite recursion: never reaches base case
- Stack overflow error

The fix: Recursive call must use a smaller input that moves toward the base case

Common Mistake 4: Stack Overflow Scenarios

- **Deep recursion:** Calling recursive function with very large input
 - Example: `factorial(100000)` creates 100,000 stack frames
 - May exceed stack size limit
- **Solution:** Use iteration for large inputs, or increase stack size
- **Better solution:** Convert to iterative version

Prevention

- Estimate maximum recursion depth
- Use iteration if depth > 1000 (rough guideline)
- Consider iterative alternative for performance-critical code

Common Mistake 5: Performance Pitfalls

Naive Recursive Fibonacci

- **fibonacci(30)** makes 2.7 million recursive calls
- Exponential time complexity: $O(2^n)$
- Takes seconds or minutes to compute

Solutions:

- Use iterative version ($O(n)$ time, $O(1)$ space)
- Use memoization (cache results, $O(n)$ time, $O(n)$ space)
- Use dynamic programming approach

Key lesson: Not all recursive solutions are efficient. Sometimes iteration is better.

Debugging Tips for Recursion

- **Add print statements:** Print parameters at start of recursive method
- **Trace the call tree:** Draw the recursive calls on paper
- **Check base case:** Verify it's reachable and correct
- **Verify recursive call:** Ensure it moves toward base case
- **Use debugger:** Step through recursive calls to see stack
- **Test with small inputs:** Start with $n=0$, $n=1$, $n=2$
- **Check return values:** Ensure recursive call returns expected value

Debugging Tips for Iteration

- **Print loop variables:** See how they change each iteration
- **Check loop condition:** Verify it will eventually become false
- **Verify initialization:** Ensure starting values are correct
- **Check update step:** Ensure loop variable moves toward termination
- **Watch for off-by-one errors:** Common in loop bounds
- **Test edge cases:** Empty input, single element, boundary values

Using AI to Understand Recursion

- **Recursion can be confusing**: AI can help explain the flow
- **Tracing recursive calls**: Ask AI to show step-by-step execution
- **Visualizing call trees**: AI can help draw the recursive structure
- **Understanding base cases**: AI can explain why base case is needed

Example prompts:

- "Trace through factorial(4) step by step, showing each recursive call"
- "Draw the call tree for fibonacci(5)"
- "Explain why we need a base case in recursive functions"

AI Prompting Strategies: Tracing Recursion

Good Prompts for Understanding Recursion

- **"Trace this recursive function step by step:"** + code
- **"Show me the call stack for:"** + recursive call
- **"Draw the recursive call tree for:"** + function call
- **"Explain what happens when this recursive function runs:"** + code
- **"What is the base case and why is it needed?"** + code

What to include:

- Your recursive code
- A specific input value
- What you're confused about

AI Prompting Strategies: Converting Between Approaches

Converting Recursive to Iterative

- **"Convert this recursive function to an iterative version:"** + code
- **"Rewrite this using a loop instead of recursion:"** + code
- **"Show me an iterative version of this recursive algorithm:"** + code

Converting Iterative to Recursive

- **"Convert this iterative function to a recursive version:"** + code
- **"Rewrite this loop using recursion:"** + code
- **"Show me a recursive version of this iterative algorithm:"** + code

AI for Debugging Recursive Code

Debugging Prompts

- **"Why does this recursive function cause a stack overflow?"** + code
- **"What's wrong with this base case?"** + code
- **"Help me fix this infinite recursion:"** + code
- **"Why isn't this recursive function returning the correct value?"** + code

Remember:

- AI can help explain, but you need to understand the concepts
- Always verify AI suggestions
- Use AI as a learning tool, not a replacement for thinking

Summary: Recursion vs. Iteration

Recursion

- Method calls itself
- Base case + recursive case
- Natural for divide-and-conquer
- Uses call stack ($O(n)$ space)
- Can be elegant but slower
- Risk of stack overflow

When to Use

- Tree/graph structures
- Divide-and-conquer problems
- Mathematical recursion
- Code clarity priority
- Limited depth problems

Iteration

- Loops repeat code
- Initialization, condition, update
- Natural for sequential processing
- Constant stack space ($O(1)$)
- Usually faster
- No stack overflow risk

When to Use

- Performance critical
- Simple repetitive tasks
- Large problem depth
- Memory constrained
- Easy debugging needed